

Integrate an External SOAP Partner

AegisAI — Enterprise AI Security & Governance Gateway. Design-time integration of one partner edge: the partner's WSDL, the SOAP call formed by hand, its HTTP binding, how it would be discovered, and how its faults are kept behind AegisAI's own contract.

Field	Value
Team ID / Group No.	7
Repository	https://github.com/neha-nupur/AegisAi
Scope	Design-time only — no running server, no live gateway, and no service of AegisAI's own is built here.
Deliverables in this document	Task 1 (Context), Task 4 (HTTP binding), Task 5 (Discovery + registry entry), Task 6 (Fault mapping)
Companion files	<code>partner.wsdl</code> (Task 2), <code>soap-request.xml</code> , <code>soap-response.xml</code> , <code>soap-fault.xml</code> (Task 3)

Team members

S. No.	Role	Name	Roll Number
1	Group Leader	Aditi Garg	20251651008
2	Group Member	Neha Nupur	20251651064
3	Group Member	Shivam Kumar Soni	20251651084

Task 1 Context: the partner edge, and why it is SOAP

Partner: Meridian Messaging Ltd. — the *Meridian SMS Notification Service*, v1.

Operation integrated: `sendSMS` — one operation, one direction, one contract.

AegisAI's approval workflow holds an AI request whenever the Policy Engine returns `REQUIRE_APPROVAL`. Holding it is only half the job: a named human has to be told, within seconds, that something is waiting for them. Meridian Messaging is the institute's SMS aggregator, it delivers those reminders to the approver's handset, and it exposes a **SOAP 1.1** interface and nothing else. That makes it AegisAI's single SOAP edge in an otherwise entirely REST system.

Why this edge is SOAP while the rest of AegisAI is REST

1. Strong contract. Meridian ships a WSDL that pins every field, its type and its cardinality — `to` must match the E.164 pattern `\+[1-9][0-9]{1,14}`, `priority` is a closed two-value enumeration, `variables` is optional and repeatable — so a breaking change on their side fails the moment AegisAI regenerates and revalidates its binding, rather than at two in the morning in production; AegisAI's own REST resources can evolve by adding fields to JSON, but when the contract belongs to somebody else it has to be frozen somewhere, and that somewhere is the WSDL.

2. Message-level security. The payload names an individual approver and the action they are being asked to authorise, so Meridian mandates a WS-Security header in which the credential, a freshness `Timestamp` and an RSA signature travel *inside* the message and remain verifiable after some intermediary has terminated TLS — end-to-end integrity and non-repudiation, which transport security alone cannot provide.

3. Transactions. `sendSMS` is a billable, non-idempotent side effect against a carrier network, and SOAP gives that a typed, contractually declared `soap:Fault` — `CARRIER_REJECTED` or `THROTTLED`, explicitly marked retryable or not — which AegisAI can

map exhaustively onto its own vocabulary, instead of having to guess what an arbitrary HTTP status from an unrelated service was meant to say.

4. Everything AegisAI owns itself — resources, payloads, caching, pagination, errors — stays REST, because there the contract is ours to evolve; only this one edge, where the contract is Meridian's, is written in WSDL.

Design boundary respected: nothing in this document is a service AegisAI exposes. AegisAI is the *consumer* here. The WSDL was authored because Meridian's was unavailable for the exercise; in production the same file is fetched from Meridian and pinned (see Task 5).

Task 4

The HTTP binding

The binding declares `style="document"`, `transport="http://schemas.xmlsoap.org/soap/http"` and `soapAction="https://api.meridian-sms.com/ws/v1/notifications/sendSMS"`; the port declares `<soap:address location="https://api.meridian-sms.com/ws/v1/notifications"/>`. Between them those two declarations determine every byte below.

The request as it goes onto the wire

```
POST /ws/v1/notifications HTTP/1.1
Host: api.meridian-sms.com
Content-Type: text/xml; charset="utf-8"
SOAPAction: "https://api.meridian-sms.com/ws/v1/notifications/sendSMS"
Content-Length: 5390
Accept: text/xml
User-Agent: AegisAI-Gateway/1.0
Connection: keep-alive

<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" ...>
  <soap:Header>
    <wsse:Security soap:mustUnderstand="1">
      <wsse:BinarySecurityToken wsu:Id="cert" ...>MIID...</wsse:BinarySecurityToken>
      <wsu:Timestamp wsu:Id="ts"> ... </wsu:Timestamp>
      <wsse:UsernameToken wsu:Id="ut"> ... PasswordDigest ... </wsse:UsernameToken>
      <ds:Signature> ... SignedInfo / SignatureValue ... </ds:Signature>
    </wsse:Security>
  </soap:Header>
  <soap:Body wsu:Id="body">
    <sms:SendSMSRequest>
      <sms:clientId>aegisai-gateway-eu1</sms:clientId>
      <sms:correlationId>5f1c9a3e-7b64-4c28-9d3a-1e0f6b2a8c11</sms:correlationId>
      <sms:to>+919876543210</sms:to>
      <sms:templateId>aegisai.approval.pending.v3</sms:templateId>
      <sms:variables> ... 4 variables ... </sms:variables>
      <sms:senderId>AEGISAI</sms:senderId>
      <sms:priority>HIGH</sms:priority>
      <sms:callbackUrl>https://hooks.aegisai.io/v1/sms-status</sms:callbackUrl>
    </sms:SendSMSRequest>
  </soap:Body>
</soap:Envelope>
```

(abbreviated above; the complete 5,390-byte body is submitted as soap-request.xml)

Where every line comes from

Request line / header	Declared in the contract	Why
POST	<code><soap:binding transport="http://schemas.xmlsoap.org/soap/http"/></code> binding, line 1	The SOAP 1.1 HTTP binding has no GET form for an operation with input; <code>sendSMS</code> always arrives as POST.
<code>/ws/v1/notifications</code> (path)	<code><soap:address location="https://api.meridian- sms.com/ws/v1/notifications"/></code> <code>wsdl:port</code>	The path component of the endpoint URL published by the <code>service/port</code> .
Host: <code>api.meridian-sms.com</code>	same <code>soap:address</code>	The authority component of the same URL. HTTP/1.1 requires it; HTTP/2 would carry it in <code>:authority</code> instead.
Content-Type: <code>text/xml; charset="utf-8"</code>	SOAP 1.1 media type (fixed by the SOAP version the binding speaks)	SOAP 1.1 is <code>text/xml</code> . Had the binding been SOAP 1.2 this line would be <code>application/soap+xml; action="..."</code> and the <code>SOAPAction</code> header would disappear into it.
SOAPAction: <code>".../notifications/sendSMS"</code>	<code><soap:operation soapAction="https://api.meridian- sms.com/ws/v1/notifications/sendSMS"/></code> <code>wsdl:binding/wsdl:operation</code>	Quoted, byte-for-byte identical to the binding. SOAP 1.1 requires the header even when empty; a mismatch is rejected before the body is parsed.
Content-Length: <code>5390</code>	the serialised envelope	Exact byte length of <code>soap-request.xml</code> , which is the body — there is no wrapper, no form encoding and no JSON on either side of it.

The whole binding in one line. The contract is document/literal, SOAP 1.1, over HTTP POST: one schema element per message, unqualified by the SOAP envelope's own namespace, so the receiver can validate the body against `wsdl:types` before any application code runs. That is the property Task 1 asked for.

Task 5**Discovery, in modern form**

AegisAI obtains the contract in one of two ways. No UDDI server is run; the registry is a record in AegisAI's own store.

(a) Direct URL — WSDL-first, pinned at build time

```
GET /ws/v1/notifications?wsdl HTTP/1.1
Host: api.meridian-sms.com
```

Meridian follows the conventional WSDL-GET reply — the WSDL itself, at a URL derived from the endpoint by appending `?wsdl`. This is how the contract is *acquired*: fetched once, hashed, and committed.

Pinning. The acquired contract is recorded as `partner.wsdl` with

sha256 a725b3810790d86444b253d7a8b7322921d040af19ce10e5d9a5fc06ffd229de

CI re-hashes the file on every build. If Meridian edits the contract without telling us, the digest changes, the build fails, and the change becomes a deliberate decision instead of a 3 a.m. incident. A loose reference to a live `?wsdl` URL is not discovery; it is a dependency on someone else's deploy schedule.

(b) Lookup — a catalogue / registry entry

Discovery at runtime is a *lookup*: AegisAI's Integration Catalogue answers "who provides this, where is it, and which contract does it speak?" In UDDI terms that answer is built from a **tModel** — the reusable *definition* of a capability, which many services can point at — and a **businessService**, the concrete published endpoint that points at it. The tModel is the important half: it decouples "what this service is" from "who currently provides it".

The tModel — a pointer to the WSDL

```
tModelKey    urn:aegisai:catalogue:tmodels:meridian-sms.wsdl.v1
name         meridian-sms.wsdl.v1 ("the WSDL this service speaks")
overviewDoc  https://api.meridian-sms.com/ws/v1/notifications?wsdl
categoryBag
  tModelSpec uddi:wsdlSpec
  types      wsdl11-soaphttp-document
```

The registry entry — one record, business → service → endpoint → WSDL

```
{
  "catalogue": "aegisai.integrations",
  "recordId": "int-2026-014",
  "registeredAt": "2026-10-07T18:21:04Z",

  "business": {
    "name": "Meridian Messaging Ltd.",
    "businessKey": "urn:aegisai:catalogue:businesses:meridian-messaging",
    "contact": "partners@meridian-sms.com"
  },

  "service": {
    "name": "Meridian SMS Notification Service",
    "serviceKey": "urn:aegisai:catalogue:services:meridian-sms-v1",
    "version": "1.0.0",
    "operations": ["sendSMS"],
    "binding": "NotificationSoap11Binding",
    "style": "document/literal",
    "protocol": "SOAP 1.1 over HTTP"
  },

  "endpoint": {
    "accessPoint": "https://api.meridian-sms.com/ws/v1/notifications",
    "endpointType": "https",
    "httpMethod": "POST",
    "soapAction": "https://api.meridian-sms.com/ws/v1/notifications/sendSMS"
  },

  "contract": {
    "wsdl": "https://api.meridian-sms.com/ws/v1/notifications?wsdl",
    "tModel": "urn:aegisai:catalogue:tmodels:meridian-sms.wsdl.v1",
    "pinnedCopy": "Assignment 3/partner.wsdl",
    "sha256": "a725b3810790d86444b253d7a8b7322921d040af19ce10e5d9a5fc06ffd229de"
  },

  "policy": {
    "security": "WS-Security: Timestamp + UsernameToken(PasswordDigest) + ds:Signature(RSA-SHA1)",
    "transport": "TLS 1.3, mutual-auth certificates pinned",
    "timeoutMs": 3000,
    "owner": "platform-integrations@aegisai.io"
  }
}
```

How a consumer resolves it

1. Ask the catalogue for `serviceKey` — or, more usually, ask for capability *SMS delivery* and let the catalogue return every provider whose `tModel` carries `types = wsdl11-soaphttp-document`.
2. Read `endpoint.accessPoint` — that is the `Host` and path of Task 4.
3. Read `contract.tModel` → `overviewDoc` to learn which WSDL the endpoint speaks, then compare `contract.sha256` against the pinned copy before binding.
4. Bind. The catalogue is consulted at deploy time; at run time AegisAI talks to the endpoint it already pinned, so a catalogue outage can never take the gateway down.

Modern form, honestly stated. This is a catalogue record, not a running UDDI registry: the classic `businessEntity` → `businessService` → `bindingTemplate` → `tModelInstanceInfo` → `tModel` chain has been flattened into one document, but every role in it survives — business, service, endpoint, and the `tModel`-style pointer that says which contract the endpoint speaks. No UDDI server is required or run, exactly as the task allows.

Task 6**Fault mapping — the partner's words stop at our border**

The partner's vocabulary must not leak to AegisAI's callers. A fault arriving from Meridian is an *internal* event; the error AegisAI returns is built exclusively from AegisAI's own contract. The mapping is many-to-few: Meridian has a dozen codes, AegisAI has a handful, and the choice of HTTP status is made by AegisAI's semantics — never inherited from the partner.

What arrives

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/xml; charset="utf-8"

<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Message could not be delivered to the terminating carrier</faultstring>
  <detail>
    <sms:SendSMSFault>
      <sms:faultCode>CARRIER_REJECTED</sms:faultCode>
      <sms:faultString>MSISDN is barred or unreachable on the terminating carrier</sms:faultString>
      <sms:retryable>false</sms:retryable>
      <sms:partnerTraceId>mm-trace-8f41c2d9</sms:partnerTraceId>
    </sms:SendSMSFault>
  </detail>
</soap:Fault>
```

Exactly the envelope submitted as `soap-fault.xml`, declared in `partner.wsdl` as fault `SendSMSFault`.

What leaves

AegisAI answers in the error envelope its own contract defines — `type` / `title` / `status` / `detail` / `code`, the same Problem-Details shape implemented by `Assignment 4/Part C - Implement It/errors.py`:

```
{
  "type": "https://aegisai.example/errors/notification_unavailable",
  "title": "Approver could not be notified",
  "status": 503,
  "detail": "The approval reminder could not be delivered. The request remains pending approval.",
  "code": "notification_unavailable",
  "correlationId": "5f1c9a3e-7b64-4c28-9d3a-1e0f6b2a8c11"
}
```

The mapping table

Partner <code>faultCode</code>	Partner's meaning (internal only)	AegisAI <code>code</code>	HTTP	Retry	What AegisAI's caller is told
CARRIER_REJECTED	MSISDN barred / unreachable on the terminating carrier	notification_unavailable	503	no — switch channel	The approval reminder could not be delivered; the request itself is still pending . AegisAI falls back to in-app + email and records a <code>SecurityEvent</code> .
INVALID_MSISDN	Recipient number malformed	configuration_invalid	500	no	The approver contact on file is not usable. The bad data is AegisAI's, so AegisAI owns the 5xx — never a 4xx blaming the caller.
THROTTLED	Partner-side rate limit / credit	rate_limited_upstream	503	yes, after <code>retryAfterSeconds</code>	Delivery is temporarily delayed; <code>Retry-After</code> is AegisAI's value, derived from — not copied as — the partner's number.
AUTH_FAILED	Meridian rejected <i>our</i> credentials	upstream_unavailable	503	no	A dependency is unavailable. Which dependency, and why, goes to the audit log and the on-call pager — never to the caller.
transport timeout / no response	Meridian never answered	upstream_unavailable	503	yes, exponential backoff	Same code as above. A timeout is not a different <i>sentence</i> , only a different retry policy.
DUPLICATE_MESSAGE	Idempotency key already consumed	conflict	409	no	This notification was already submitted — AegisAI's own idempotency contract, whose code already exists in <code>errors.py</code> .

The rule, stated explicitly

These strings are generated by AegisAI and must never appear in an AegisAI response body:

```
CARRIER_REJECTED · INVALID_MSISDN · THROTTLED · AUTH_FAILED · DUPLICATE_MESSAGE
MSISDN · "terminating carrier" · Meridian · meridian-sms.com
mm-trace-8f41c2d9 · soap:Client · .../ws/sms/v1
```

They are AegisAI's upstream's vocabulary. A caller that depends on them would be depending on a vendor it has no contract with — and swapping Meridian for another aggregator would break callers AegisAI never told about Meridian in the first place.

Where the partner's words do go

Diagnosability is preserved without leaking anything. The gateway writes one audit record per attempt, keyed by the AegisAI correlationId that already appears in the request's sms:correlationId field:

Audit field (internal)	Value from the fault above
correlationId	5f1c9a3e-7b64-4c28-9d3a-1e0f6b2a8c11 (AegisAI's, also returned to the caller)
upstream.vendor	meridian-messaging
upstream.faultCode	CARRIER_REJECTED
upstream.faultString	MSISDN is barred or unreachable...
upstream.partnerTraceId	mm-trace-8f41c2d9 (quoted to Meridian support, never to a caller)
upstream.retryable	false → AegisAI's own retry policy, not the caller's

So the chain is: caller → AegisAI's correlationId → audit record → the partner's full fault. An operator with the correlation ID can reconstruct everything; a caller sees only a stable, versioned AegisAI error it was never told it had to understand. That is the whole point of the boundary — and it is the same reason Assignment 1 insisted on 401, 404 and 422 staying distinct: an error vocabulary is a contract, and a contract only holds if exactly one party owns it.

Appendix A — files submitted with this document

File	Task	Contents
integration.pdf	1, 4, 5, 6	This document: context, HTTP binding, discovery + registry entry, fault mapping, team details.
partner.wsdl	2	The partner contract — all six WSDL 1.1 elements: <code>types</code> , <code>message</code> ×3, <code>portType</code> , <code>binding</code> (SOAP 1.1 over HTTP), <code>service</code> / <code>port</code> with endpoint URL.
soap-request.xml	3	Request: WS-Security credentials in the Header, <code>SendSMSRequest</code> in the Body. 5,390 bytes.
soap-response.xml	3	Success response: <code>SendSMSResponse</code> — <code>ACCEPTED</code> , message ID, segments, timestamp.
soap-fault.xml	3	<code>soap:Fault</code> carrying <code>SendSMSFault</code> : <code>CARRIER_REJECTED</code> , not retryable.
verification.txt	—	Machine-generated proof that the above agree (see Appendix B).

Appendix B — the messages were checked, not asserted

Every claim in this document was verified by re-reading the submitted files from disk. The run recorded in `verification.txt` reports **27 checks passed, 0 failed**: the WSDL parses and all six elements are present with every internal reference resolving; the schema lifts cleanly out of `wsdl:types` as valid XSD; **all three envelopes validate against that schema**; the request's child order matches the declared `xsd:sequence` exactly; both WS-Security `DigestValues` recompute from exclusive-C14N + SHA-1; the RSA-SHA1 signature verifies against the certificate embedded in the message itself; and the `PasswordDigest` recomputes as `Base64(SHA1(nonce + created + password))`.

Check	Result
<code>wsdl:types</code> , <code>message</code> ×3, <code>portType</code> , <code>binding</code> , <code>service</code> , <code>port</code> present; all references resolve	PASS
<code>SOAPAction</code> = <code>https://api.meridian-sms.com/ws/v1/notifications/sendSMS</code>	PASS
endpoint from <code>service/port</code> = <code>https://api.meridian-sms.com/ws/v1/notifications</code>	PASS
<code>SendSMSRequest</code> , <code>SendSMSResponse</code> , <code>SendSMSFault</code> validate against the WSDL schema	PASS ×3
request child order == declared <code>xsd:sequence</code>	PASS
both reference digests recompute (exc-c14n + SHA-1)	PASS ×2
signedInfo RSA-SHA1 verifies against the embedded certificate	PASS
<code>PasswordDigest</code> recomputes	PASS
<code>soap:Fault</code> / <code>faultcode</code> / <code>detail</code> structure	PASS ×3